

Semester	B.E. Semester VII – CMPN
Subject	R Programming LAB
Subject Professor In-charge	Prof. Sanjeev Dwivedi
Assisting Teachers	
Laboratory	M411

Student Name	Eshanika Amballa
Roll Number	23102B0056
Grade and Subject Teacher's Signature	

Experiment Number	03
Experiment Title	Image recognition using R
Program	<pre>library(reticulate) # tell reticulate this script needs TensorFlow py_require("tensorflow") library(EBImage) library(keras3) # read images setwd('C:/Users/Eshan/Downloads/images') pics <- c('p1.jpg','p2.jpg','p3.jpg','p4.jpg','p5.jpg','p6.jpg', 'c1.jpg','c2.jpg','c3.jpg','c4.jpg','c5.jpg','c6.jpg') mypic <- list() for (i in 1:12) { mypic[[i]] <- readImage(pics[i]) } # explore for (i in 1:12) { display(mypic[[i]]) } # Optional single-window look at all 12 at once instead of 12 pop-ups # display(combine(mypic), method = "raster", all = TRUE)</pre>

```

print(mypic[[1]])      # dimensions, color mode, storage type
summary(mypic[[1]])
# hist(mypic[[1]])
str(mypic)

# resize every image to the same size
# 28x28 keeps this in line with the tutorial (fast to train), but with
only
# 12 source photos you can bump this up (e.g. 64) later without
changing
# the rest of the script logic.
for (i in 1:12) { mypic[[i]] <- resize(mypic[[i]], 28, 28) }

# reshape each image into a 28 x 28 x 3 array (RGB)
for (i in 1:12) { mypic[[i]] <- array_reshape(mypic[[i]], c(28, 28, 3))
}

# build train/test sets
# 5 planes + 5 cars for training, 1 plane + 1 car held out for
testing.
# rbind() on a (28,28,3) array flattens it to a single row of length
# 28*28*3 = 2352 that's why the first dense layer below uses
input_shape = c(2352).
trainx <- NULL
for (i in 1:5) { trainx <- rbind(trainx, mypic[[i]]) } # p1-p5
for (i in 7:11) { trainx <- rbind(trainx, mypic[[i]]) } # c1-c5

testx <- rbind(mypic[[6]], mypic[[12]])              # p6, c6

# labels: 0 = plane, 1 = car
trainy <- c(0,0,0,0,0, 1,1,1,1,1)
testy <- c(0, 1)

trainLabels <- to_categorical(trainy)
testLabels <- to_categorical(testy)

# build the model
model <- keras_model_sequential()
model %>%
  layer_dense(units = 256, activation = 'relu', input_shape =
c(2352)) %>%
  layer_dense(units = 128, activation = 'relu') %>%
  layer_dense(units = 2, activation = 'softmax')

```

```
summary(model)

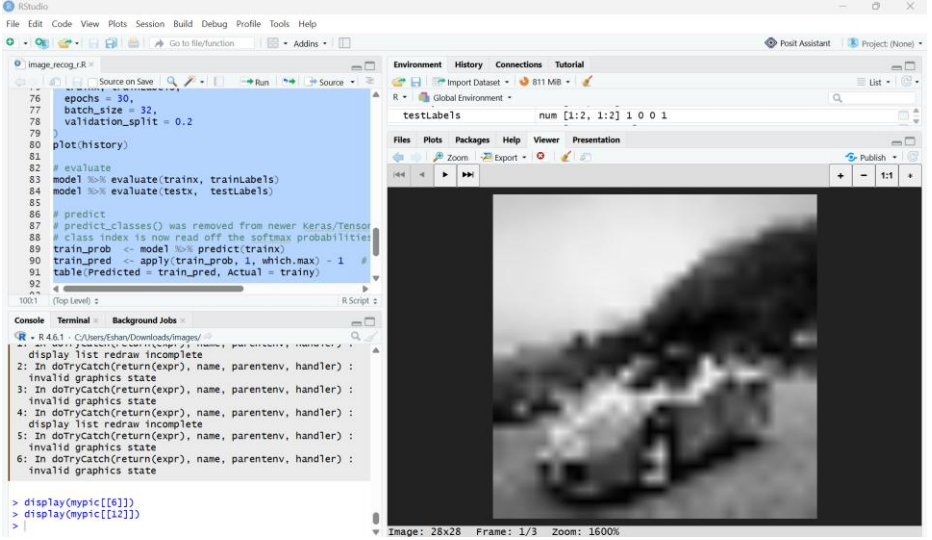
# Compile
model %>% compile(
  loss = 'binary_crossentropy', # matches the tutorial;
  'categorical_crossentropy'
  optimizer = optimizer_rmsprop(), # is the more textbook-
  correct choice for one-hot
  metrics = 'accuracy'           # labels + softmax, and works
  just as well here
)

# train
# with only 10 training images this is a toy example
validation_split = 0.2
# leaves just 2 samples for validation, and accuracy will bounce
around a lot.
# It's meant to show the workflow end-to-end, not to produce a
production model.
history <- model %>% fit(
  trainx, trainLabels,
  epochs = 30,
  batch_size = 32,
  validation_split = 0.2
)
plot(history)

# evaluate
model %>% evaluate(trainx, trainLabels)
model %>% evaluate(testx, testLabels)

# predict
# predict_classes() was removed from newer Keras/TensorFlow
releases, so the
# class index is now read off the softmax probabilities directly.
train_prob <- model %>% predict(trainx)
train_pred <- apply(train_prob, 1, which.max) - 1 # -1 to convert
to 0/1 label
table(Predicted = train_pred, Actual = trainy)

test_prob <- model %>% predict(testx)
test_pred <- apply(test_prob, 1, which.max) - 1
table(Predicted = test_pred, Actual = testy)
```

	# class probabilities, if you want to see confidence rather than just the label train_prob test_prob
Output	 The screenshot displays the RStudio interface. The script editor contains the following code: <pre> epochs = 30, batch_size = 32, validation_split = 0.2) plot(history) # evaluate model %>% evaluate(trainx, trainLabels) model %>% evaluate(testx, testLabels) # predict # predict_classes() was removed from newer keras/TensorFlow # class index is now read off the softmax probabilities train_prob <- model %>% predict(trainx) train_pred <- apply(train_prob, 1, which.max) - 1 table(Predicted = train_pred, Actual = trainy) </pre> The Environment pane shows the following objects: - Global Environment: num [1:2, 1:2] 1 0 0 1 - testLabels: num [1:2, 1:2] 1 0 0 1 The Console shows the following output: <pre> > display(mypic[[6]]) > display(mypic[[12]]) > </pre>

